

Requisitos de Segurança - Projeto *Mãos em ação*

Mecanismo 1 – Controle de Acesso por Perfil (RBAC)

OWASP: A01:2021 – Broken Access Control, refere-se a falhas no controle de acesso que permitem que usuários executem ações ou acessem recursos sem a devida autorização.

Requisito de Segurança: somente usuários com perfil *ONG* podem criar, editar ou excluir oportunidades de voluntariado. Usuários com perfil *Voluntário* podem apenas visualizar oportunidades e realizar inscrições.

Risco: um usuário sem autorização pode acessar funcionalidades restritas do sistema e executar operações que não deveria.

Por que esse risco importa: caso um voluntário consiga acessar funções exclusivas das ONGs, ele poderá alterar ou excluir oportunidades de voluntariado, comprometendo a integridade do sistema e prejudicando as organizações cadastradas.

Como foi mitigado: foi implementado um controle de acesso baseado em papéis (RBAC), onde cada usuário possui um perfil. Antes de executar operações críticas, o backend verifica se o usuário possui a permissão necessária.

Onde aparece no código: No views.py:

```
33 def volunteer_required(view_func):
34     """
35     RBAC: Enforces that the user must have a Volunteer profile.
36     """
37     @wraps(view_func)
38     @api_login_required
39     def wrapped_view(request, *args, **kwargs):
40         try:
41             if request.user.profile.role != 'VOLUNTEER':
42                 return JsonResponse({'error': 'Acesso negado. Apenas voluntários podem realizar esta ação.'}, status=403)
43         except UserProfile.DoesNotExist:
44             return JsonResponse({'error': 'Perfil de usuário não configurado.'}, status=403)
45         return view_func(request, *args, **kwargs)
46     return wrapped_view
47
48
49 def ngo_required(view_func):
50     """
51     RBAC: Enforces that the user must have an NGO profile.
52     """
53     @wraps(view_func)
54     @api_login_required
55     def wrapped_view(request, *args, **kwargs):
56         try:
57             if request.user.profile.role != 'NGO':
58                 return JsonResponse({'error': 'Acesso negado. Apenas ONGs podem realizar esta ação.'}, status=403)
59         except UserProfile.DoesNotExist:
60             return JsonResponse({'error': 'Perfil de usuário não configurado.'}, status=403)
61         return view_func(request, *args, **kwargs)
62     return wrapped_view
63
```

CWE: CWE-862 – Missing Authorization, o sistema não verifica adequadamente se o usuário possui permissão para executar determinada ação.

CVE de Referência: CVE-2021-22986, vulnerabilidade que permitia contornar controles de acesso em determinadas condições, possibilitando acesso indevido a recursos protegidos.

DFD:

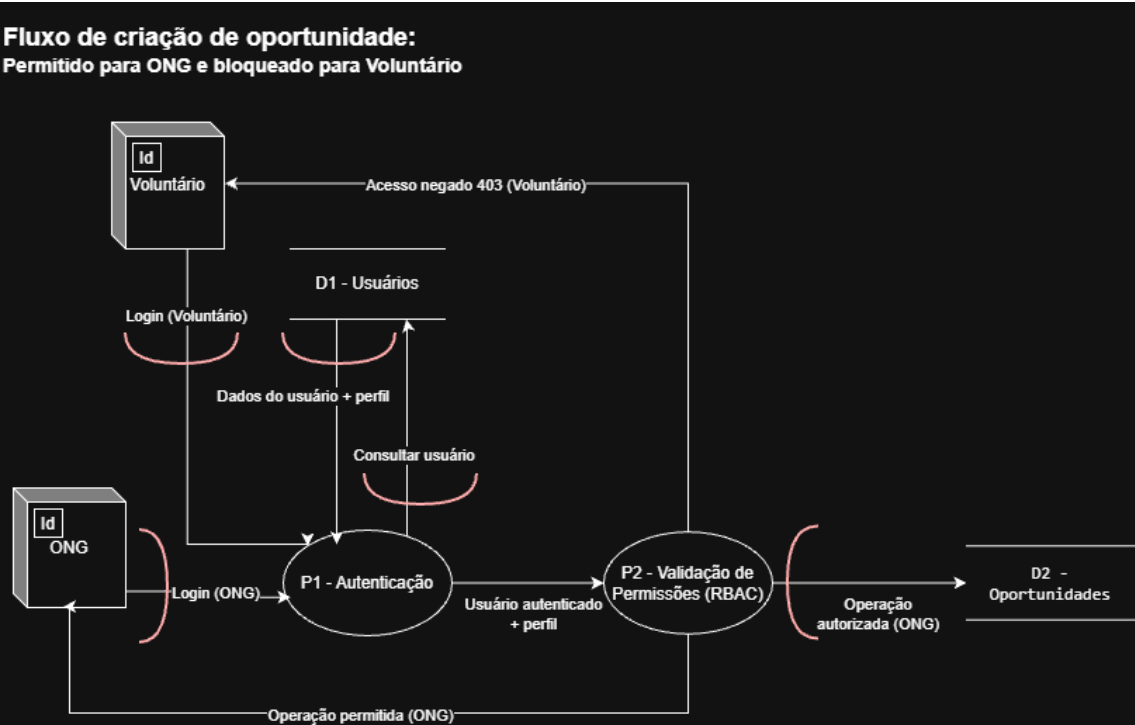


Tabela STRIDE:

Categoria STRIDE	Fluxo Afetado	Possível Ameaça	Mitigação Aplicada
Spoofing	Login	Usuário tenta se passar por outro usuário	Login + senha + 2FA
Tampering	Requisições para criação de oportunidade	Usuário altera parâmetros enviados ao backend	Validação no backend
Repudiation	Alteração de oportunidades	Usuário nega ter realizado uma ação	Log de auditoria
Information Disclosure	Consulta de oportunidades e usuários	Usuário acessa dados sem autorização	Controle de acesso RBAC
Denial of Service	Processo de autenticação	Tentativas massivas de login	Limitação de tentativas
Elevation of Privilege	Controle de permissões	Voluntário tenta executar funções de ONG	RBAC e validação de perfil no backend

Mecanismo 2 – Proteção contra SQL Injection

OWASP: A03:2021 – Injection, ocorre quando dados fornecidos pelo usuário são interpretados como comandos, permitindo a execução de instruções maliciosas.

Requisito de Segurança: todas as consultas ao banco de dados devem utilizar consultas parametrizadas.

Risco: um atacante envia comandos SQL maliciosos através dos campos de entrada da aplicação.

Por que esse risco importa: a exploração pode permitir acesso indevido a informações, alteração de registros ou até exclusão completa dos dados armazenados.

Como foi mitigado: o sistema utiliza consultas parametrizadas e validação de entradas, impedindo que dados fornecidos pelo usuário sejam interpretados como comandos SQL.

Onde	aparece	no	código:	No	arquivo	Views.py:
235			<code>vacancies = Vacancy.objects.all().select_related('ngo_profile_user').order_by('-created_at')</code>			
273			<code>vacancy = Vacancy.objects.create(</code>			
298			<code>vacancy = get_object_or_404(Vacancy, id=vacancy_id)</code>			
451			<code>app = get_object_or_404(Application, id=application_id)</code>			

CWE: CWE-89 – Improper Neutralization of Special Elements used in an SQL Command, falha causada pela ausência de tratamento adequado de entradas utilizadas em consultas SQL.

CVE de Referência: CVE-2022-22965, vulnerabilidade amplamente divulgada que permitia a execução remota de comandos através da exploração de entradas não tratadas adequadamente.

Mecanismo 3 – Autenticação Multifator (MFA / 2FA)

OWASP: A07:2021 – Identification and Authentication Failures, vulnerabilidade amplamente divulgada que permitia a execução remota de comandos através da exploração de entradas não tratadas adequadamente.

Requisito de Segurança: o sistema deve exigir um segundo fator de autenticação após a validação da senha.

Risco: credenciais roubadas podem ser utilizadas para acessar contas legítimas.

Por que esse risco importa: mesmo utilizando senhas fortes, usuários podem sofrer phishing, vazamento de dados ou reutilização de senhas.

Como foi mitigado: após a validação da senha, o sistema solicita um código temporário enviado ao usuário. O acesso somente é concedido após a validação do segundo fator.

Onde aparece no código: No Settings.py:

```
183 SOCIALACCOUNT_PROVIDERS = {
184     'amazon_cognito': {
185         'APP': {
186             'client_id': COGNITO_APP_CLIENT_ID,
187             'secret': COGNITO_APP_CLIENT_SECRET,
188             'key': '',
189         },
190         'DOMAIN': COGNITO_DOMAIN,
191     },
192 }
193
```

CWE: CWE-287 – Improper Authentication, o sistema não autentica corretamente os usuários antes de conceder acesso aos recursos.

CVE de Referência: CVE-2019-6111, vulnerabilidade que explorava falhas de autenticação e validação de identidade durante a comunicação entre sistemas.

Mecanismo 4 – Validação de Regras de Negócio (Design Inseguro)

OWASP: A04:2021 – Insecure Design, refere-se a falhas no próprio projeto da aplicação, permitindo comportamentos inseguros mesmo quando o código não possui erros técnicos.

Requisito de Segurança: o sistema deve validar regras de negócio críticas para impedir comportamentos indevidos ou inconsistentes, mesmo quando um usuário autenticado tenta manipular requisições enviadas ao servidor. Exemplos de Regras Implementadas:

- Um voluntário não pode se inscrever duas vezes na mesma oportunidade.
- Uma oportunidade não pode receber mais inscritos do que o número de vagas disponíveis.
- Uma ONG não pode editar ou excluir oportunidades criadas por outra organização.

Risco: o sistema pode parecer funcional, mas permitir ações indevidas devido a falhas na sua concepção e modelagem (lógica de negócio), independentemente da qualidade da escrita do código.

Por que esse risco importa: usuários autenticados podem explorar falhas de lógica para burlar o fluxo esperado. Isso permite que um voluntário monopolize vagas via múltiplas inscrições ou que uma ONG altere dados de terceiros, comprometendo a integridade do sistema.

Como foi mitigado: implementação de validações severas diretamente no backend antes de qualquer operação crítica. O sistema agora barra a ação e retorna um erro caso o voluntário já esteja inscrito, as vagas estejam esgotadas ou a oportunidade não pertença à ONG requisitante.

Onde aparece no código: No models.py:

```
85 # Business Rule: A volunteer can only apply to a vacancy once.  
86 unique_together = ('volunteer', 'vacancy')
```

No Views.py:

```
367 # 🛡️ Validação de Regra de Negócio (Design Inseguro)  
368 # Evita que o mesmo voluntário crie mais de uma candidatura para a mesma vaga  
369 try:  
370     Application.objects.create(  
371         volunteer=volunteer_profile,  
372         vacancy=vacancy,  
373         status='PENDING'  
374     )  
375 except IntegrityError:  
376     return JsonResponse({'error': 'Você já se candidatou a esta vaga.'}, status=400)  
377  
378     return JsonResponse({'message': 'Candidatura enviada com sucesso!'}, status=201)  
379
```

```
317 try:  
318     profile = request.user.profile  
319     if profile.role != 'NGO' or vacancy.ngo != profile.ngo:  
320         return JsonResponse({'error': 'Acesso negado. Apenas a ONG criadora desta vaga pode alterá-la.'}, status=403)  
321 except Exception:  
322     return JsonResponse({'error': 'Acesso negado.'}, status=403)
```

```
341 try:  
342     profile = request.user.profile  
343     if profile.role != 'NGO' or vacancy.ngo != profile.ngo:  
344         return JsonResponse({'error': 'Acesso negado. Apenas a ONG criadora desta vaga pode deletá-la.'}, status=403)  
345 except Exception:  
346     return JsonResponse({'error': 'Acesso negado.'}, status=403)  
347
```

CWE: CWE-840 – Business Logic Errors, falhas relacionadas às regras de negócio da aplicação que podem ser exploradas por usuários para obter vantagens indevidas.

CVE de Referência: CVE-2023-34362, vulnerabilidade que explorava falhas de lógica e validação de processos internos para acessar funcionalidades não previstas.

Mecanismo 5 – Criptografia de Dados Sensíveis

OWASP: A02:2021 – Cryptographic Failures, relaciona-se ao uso inadequado ou à ausência de mecanismos de criptografia para proteger informações sensíveis.

Requisito de Segurança: informações sensíveis dos usuários devem permanecer protegidas quando armazenadas no banco de dados.

Risco: caso o banco seja comprometido, dados pessoais podem ser expostos.

Por que esse risco importa: a exposição de dados pessoais pode gerar danos aos usuários, violações de privacidade e problemas legais relacionados à LGPD.

Como foi mitigado: foi implementada criptografia híbrida. Os dados sensíveis são criptografados utilizando AES, enquanto a chave AES é protegida por RSA.

Onde aparece no código: No models.py:

```
43     cpf = EncryptedCharField(blank=True, null=True)
44     phone = EncryptedCharField(blank=True, null=True)

52     profile = models.OneToOneField(UserProfile, on_delete=models.CASCADE, related_name='ngo')
53     cnpj = EncryptedCharField(blank=True, null=True)
54     phone = EncryptedCharField(blank=True, null=True)
55     address = EncryptedCharField(blank=True, null=True)
```

CWE: CWE-311 – Missing Encryption of Sensitive Data, dados confidenciais são armazenados ou transmitidos sem proteção criptográfica adequada.

CVE de Referência: CVE-2017-5638, vulnerabilidade que resultou na exposição de informações sensíveis devido a falhas de proteção e tratamento de dados.